

NP-Completeness

The P vs. NP Question, Polynomial-Time
Reductions, and Engineering Judgment

CSE 4809 — Algorithm Engineering

August 4, 2026

Roadmap

- 1 Why some problems resist every algorithm we throw at them
- 2 The classes P and NP, and what a *certificate* is
- 3 Polynomial-time reductions — the one tool that does all the work
- 4 The first NP-complete problem, and the reduction chain it spawns
- 5 Two full worked reductions: 3-CNF-SAT $\rightarrow$ CLIQUE $\rightarrow$ VERTEX-COVER
- 6 Recognizing NP-hardness on the job, and what to do about it

By the end of this lecture

You should be able to *spot* a likely NP-hard problem in the wild, sketch a reduction to justify that suspicion, and choose a sensible engineering response instead of chasing an exact polynomial-time algorithm that (probably) does not exist.

Why Hardness Matters

Section 1 — 0:00 – 0:10

Some pairs of problems look almost identical on the surface. One sits comfortably in polynomial time; the other has resisted every algorithm ever tried against it.

Solvable in poly. time

Shortest simple path between two vertices

Euler tour (visit every *edge* once)

2-CNF satisfiability

No known poly. algorithm

Longest simple path between two vertices

Hamiltonian cycle (visit every *vertex* once)

3-CNF satisfiability

Key Idea

A one-word change in the problem statement can flip you from “routine coursework” to “open research question.” Telling the two apart is the point of this lecture.

Tractable

A problem is **tractable** if some algorithm solves every instance of size n in $O(n^k)$ time, for a constant k independent of the input.

Intractable

A problem is (believed) **intractable** if every known algorithm requires superpolynomial time — e.g. $2^{\Theta(n)}$ or $n!$ — in the worst case.

Where today fits

We are *not* trying to prove intractability is absolute — nobody can, yet. We build strong *evidence* of intractability via a chain of reductions, so you can stop chasing an exact fast algorithm that almost certainly does not exist.

P, NP, and Verification

Section 2 — 0:10 – 0:24

P

P is the set of decision problems for which *some* algorithm produces the correct yes/no answer in $O(n^k)$ time, for a constant k , on every input of size n .

- Almost everything covered so far this semester — MST, shortest paths, max-flow, DP formulations — lives in P .
- “Solvable” here means *from scratch*: no hints, no pre-computed help. You are handed the raw instance and must produce the answer yourself.

Suppose a colleague hands you a *claimed* solution instead of asking you to find one. How hard is it to *check* their claim?

Certificate

A **certificate** for a problem instance is a piece of evidence that, if correct, lets you confirm a “yes” answer quickly — without having to search for that evidence yourself.

Running example: Hamiltonian Cycle

- **Solving** “Does graph G have a Hamiltonian cycle?” from scratch: no known polynomial algorithm.
- **Verifying** a claimed cycle $\langle v_1, v_2, \dots, v_n \rangle$: check it visits every vertex exactly once and that consecutive vertices are adjacent — $O(n^2)$ time, easy.

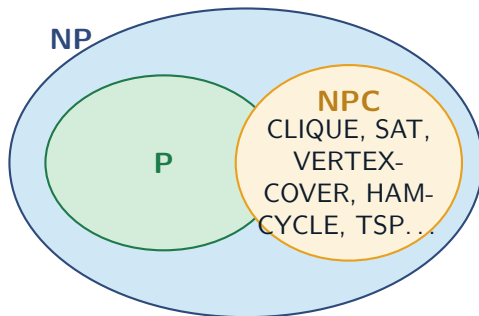
NP

NP is the set of decision problems for which a “yes” answer admits a certificate that can be **verified** in polynomial time by some algorithm $A(x, y)$, where x is the instance and y is the certificate.

- NP historically stands for *nondeterministic polynomial time* — but “polynomial-time verifiable” is the version you should actually reason with.
- Note the asymmetry: nothing is required for “no” instances beyond *not* having a fooling certificate.

Easy but important fact

$P \subseteq NP$. If you can *solve* a problem in polynomial time, you can certainly *verify* it in polynomial time — just ignore whatever certificate you’re handed and solve it yourself.



- **NPC** = the NP-complete problems: the “hardest” problems inside NP.
- Whether $P = NP$ — i.e., whether that green circle secretly equals the whole blue ellipse — is the most famous open problem in computer science. Nobody knows.
- Most theoretical computer scientists *believe* $P \neq NP$, but belief is not proof.

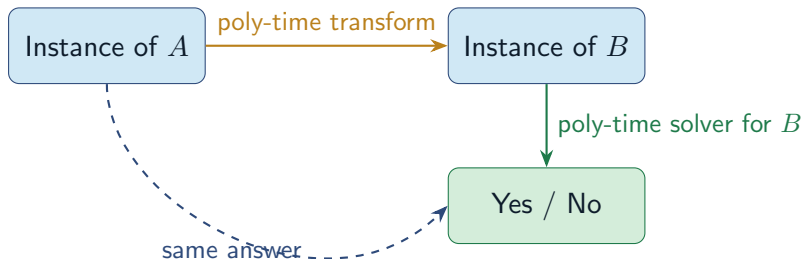
- If *any single* NP-complete problem were solved in polynomial time, **every** problem in NP would fall too — that is what “complete” means.
- Nobody has found such an algorithm despite decades of trying across thousands of NP-complete problems.
- **Engineering takeaway:** once you can show a problem is NP-complete, that is strong, community-validated evidence you should stop searching for an exact fast algorithm and pivot strategy — more on this in Section 7.

Polynomial-Time Reductions

Section 3 — 0:24 – 0:38

The core idea

If you already know how to solve problem B , and you can *transform* any instance of problem A into an instance of B — quickly, and without changing the answer — then you can solve A too: transform, solve B , hand back the answer.



Reduction $L_1 \leq_P L_2$

Language L_1 is polynomial-time reducible to L_2 , written $L_1 \leq_P L_2$, if there is a polynomial-time computable function f such that for every string x :

$$x \in L_1 \iff f(x) \in L_2.$$

- f is the **reduction function**; an algorithm computing it is a **reduction algorithm**.
- Read $L_1 \leq_P L_2$ as: L_1 is no harder than L_2 , up to a polynomial-time overhead.

If $L_1 \leq_P L_2$ and $L_2 \in P$, then $L_1 \in P$.

- *Proof idea:* transform x to $f(x)$ (poly time), run B 's poly-time algorithm on $f(x)$, return its answer. Two polynomials composed is still a polynomial.

We now flip the lemma around. Instead of using an *easy* B to solve A , we use a *known-hard* A to prove B is hard.

NP-hard / NP-complete

A language L is **NP-hard** if $L' \leq_P L$ for every $L' \in NP$.

L is **NP-complete** if, in addition, $L \in NP$.

Theorem 34.4 (stated, proof skipped)

If *any* NP-complete problem has a polynomial-time algorithm, then $P = NP$. Equivalently: if $P \neq NP$, then *no* NP-complete problem has a polynomial-time algorithm.

This is why proving a problem NP-complete is such strong evidence of intractability — it ties that one problem's fate to *every other problem in NP simultaneously*.

To prove a language L is NP-complete

- 1 Show $L \in NP$ (exhibit a poly-time verifiable certificate).
- 2 Show L is NP-hard:
 - 1 Pick a language L' *already known* to be NP-complete.
 - 2 Give a poly-time function f mapping instances of L' to instances of L .
 - 3 Prove $x \in L' \iff f(x) \in L$.
 - 4 Prove f runs in polynomial time.

This is far cheaper than reducing from *every* language in NP directly — and it's exactly what we'll do twice today.

Warning

To prove Y is NP-hard, you reduce a *known-hard* problem X **into** Y (i.e. $X \leq_P Y$) — *not* the other way around. Reducing Y into X only tells you Y is no harder than something already known to be hard, which proves nothing about Y 's hardness.

- Also remember: showing L is NP-hard alone is *not enough* for NP-completeness — you still need $L \in NP$ (Step 1 of the recipe). It's easy to forget this half.

The First NP-Complete Problem

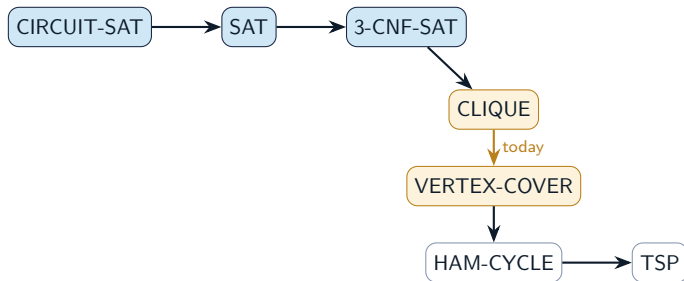
Section 4 — 0:38 – 0:45

- Our recipe always reduces *from* a problem already known to be NP-complete. That's circular unless some problem is proven hard *directly*, from the definition of NP itself.
- That first problem is **CIRCUIT-SAT**: given a boolean combinational circuit (AND / OR / NOT gates), does some input assignment make the output 1?
- Cook (1971) and, independently, Levin proved CIRCUIT-SAT (and formula satisfiability) is NP-complete by directly simulating *any* polynomial-time verification algorithm as a circuit. We will cite this result rather than re-derive it — it's a full lecture in its own right.

Key Idea

Once one problem is nailed down as NP-complete, every subsequent proof only needs *one* reduction, not a simulation argument.

Every classical NP-completeness result branches off this chain — each arrow is a poly-time reduction proved once, decades ago.



- **Today, in full:** 3-CNF-SAT $\rightarrow$ CLIQUE $\rightarrow$ VERTEX-COVER (amber).
- VERTEX-COVER $\rightarrow$ HAM-CYCLE $\rightarrow$ TSP exist in the text (gadget-heavy) — flagged as optional reading, not reproduced today.

Quick self-check before we build a real proof

- What is the difference between *solving* and *verifying* a problem?
- Why does $L_1 \leq_P L_2$ with $L_2 \in P$ imply $L_1 \in P$, but *not* the reverse implication?
- Why is it circular to always reduce “from a known NP-complete problem” — and how does CIRCUIT-SAT break that circularity?

Next: our first full proof, start to finish.

Reduction I: 3-CNF-SAT $\rightarrow$ CLIQUE

Section 5 — 0:45 – 1:05

3-CNF-SAT

Given a boolean formula $\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_k$, where every clause C_r is the OR of exactly three distinct literals, does some assignment to the variables make φ evaluate to 1?

CLIQUE

Given an undirected graph $G = (V, E)$ and integer k , does G contain a **clique** of size k — a set of k vertices that are pairwise adjacent?

- Naive CLIQUE algorithm: check all $\binom{|V|}{k}$ subsets — polynomial only when k is a fixed constant, not in general.

Certificate

The k -vertex subset $V' \subseteq V$ itself.

Verification

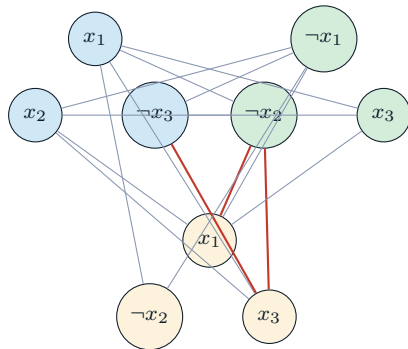
Check $|V'| = k$, then check that every pair of vertices in V' is connected by an edge — $O(k^2)$ time, clearly polynomial.

That handles half the NP-completeness recipe. Now the hard half: NP-hardness, by reduction from 3-CNF-SAT.

Given $\varphi = C_1 \wedge \cdots \wedge C_k$ in 3-CNF, build a graph G so that *a satisfying assignment corresponds exactly to a size- k clique*.

- **One triple of vertices per clause.** Clause C_r with literals $\ell_1^r, \ell_2^r, \ell_3^r$ contributes three vertices v_1^r, v_2^r, v_3^r .
- **Edges connect “compatible” literals across different clauses:** put an edge between v_i^r and v_j^s exactly when
 - 1 $r \neq s$ (different clauses), **and**
 - 2 the literals ℓ_i^r and ℓ_j^s are *not* negations of each other.
- No edges *within* a triple — a clique can therefore pick at most one vertex per clause.

$$\varphi = \underbrace{(x_1 \vee x_2 \vee \neg x_3)}_{C_1} \wedge \underbrace{(\neg x_1 \vee \neg x_2 \vee x_3)}_{C_2} \wedge \underbrace{(x_1 \vee \neg x_2 \vee x_3)}_{C_3}$$



A satisfying assignment $x_1=0, x_2=0, x_3=1$ makes $\neg x_3$ (in C_1)... wait — pick the literals that evaluate to 1: x_2 is false so use $\neg x_2$; here we select one true literal per clause: $\neg x_3 \rightarrow$ false, so instead take x_2 false $\Rightarrow$ choose $x_1=0, x_2=0, x_3=1$ gives true literals $\neg x_2$ (in C_2, C_3) and x_3 (in C_2). The **highlighted triangle** is one size-3 clique — one vertex per clause, all pairwise compatible.

Satisfying assignment $\Rightarrow$ size- k clique

- Suppose φ has a satisfying assignment.
- Every clause C_r has at least one literal that evaluates to 1 under it — pick one such literal per clause, giving one vertex per triple, k vertices total.
- Take any two chosen vertices v_i^r, v_j^s with $r \neq s$. Both corresponding literals are true simultaneously, so neither can be the negation of the other.
- By construction, that means the edge $(v_i^r, v_j^s) \in E$. This holds for every pair $\Rightarrow$ the k chosen vertices form a clique.

Size- k clique $\Rightarrow$ satisfying assignment

- Suppose G has a clique V' of size k .
- Since no edges exist inside a triple, V' contains *exactly one* vertex from each of the k triples.
- Set each corresponding literal to 1 (assign arbitrary values to any variable that appears in no chosen literal).
- Because V' is a clique, every pair of chosen literals is connected — meaning no chosen literal is the negation of another chosen literal. The assignment is therefore consistent.
- Every clause has its representative literal set to 1 $\Rightarrow$ every clause is satisfied $\Rightarrow \varphi$ is satisfied.

Running time of the reduction

- $3k$ vertices, one triple per clause.
- At most $O(k^2)$ candidate vertex pairs to test for the edge condition — each test is $O(1)$.
- Total construction time: $O(k^2)$ — polynomial in the size of φ . ✓

All four proof obligations from our recipe are now discharged: $\text{CLIQUE} \in \text{NP}$, and $3\text{-CNF-SAT} \leq_P \text{CLIQUE}$.

∴ CLIQUE is NP-complete.

Why this reduction is a good template

- The reduction must handle *any* 3-CNF formula — we cannot assume convenient structure on the A side.
- But the CLIQUE instances we *produce* are highly structured: vertices come in disjoint triples, edges only cross triples. That's completely fine — we only need to cover *some* instances of B , not all of them.
- This “general input, structured output” pattern reappears in essentially every reduction you'll write.

Reduction II: CLIQUE $\rightarrow$ VERTEX-COVER

Section 6 — 1:05 – 1:16

VERTEX-COVER

Given undirected graph $G = (V, E)$ and integer k , does there exist $V' \subseteq V$ with $|V'| \leq k$ such that every edge in E has *at least one* endpoint in V' ?

Certificate for NP membership

The set V' itself; verify $|V'| \leq k$ and check every edge against V' in $O(|E|)$ time. So VERTEX-COVER $\in$ NP.

Now for the hard half: reduce CLIQUE into VERTEX-COVER.

Graph complement

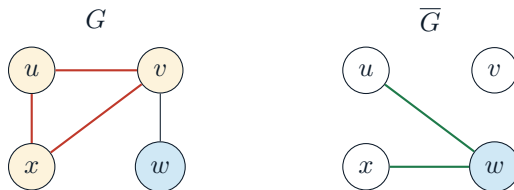
For $G = (V, E)$, the complement $\overline{G} = (V, \overline{E})$ contains exactly the edges *absent* from G .

The reduction

Given instance $\langle G, k \rangle$ of CLIQUE, output the VERTEX-COVER instance $\langle \overline{G}, |V| - k \rangle$. Computing $\overline{G}$ takes $O(|V|^2)$ time — polynomial.

Intuition

A set of vertices is a clique in G exactly when the *remaining* vertices form a vertex cover in $\overline{G}$: every edge missing from the clique's neighborhood in G shows up as an edge in $\overline{G}$ that must be covered from outside.



- G has clique $\{u, v, x\}$ of size $k = 3$ (red triangle).
- $\overline{G}$ has vertex cover $\{w\}$ of size $|V| - k = 4 - 3 = 1$ (blue) — it touches both green edges.

Clique of size k in $G \Rightarrow$ vertex cover of size $|V| - k$ in $\overline{G}$

Let V' be a clique in G . Take any edge $(u, v) \in \overline{E}$. Since $(u, v) \notin E$, at least one of u, v is *outside* V' (otherwise both would be in the clique and hence adjacent in G). So $V \setminus V'$ touches every edge of $\overline{G}$ — it's a vertex cover, and $|V \setminus V'| = |V| - k$.

Vertex cover of size $|V| - k$ in $\overline{G} \Rightarrow$ clique of size k in G

Run the same argument in reverse: if V' covers $\overline{G}$, then no edge of $\overline{G}$ has both endpoints in $V \setminus V'$ — meaning every pair in $V \setminus V'$ is adjacent in G . That's a clique of size $|V| - |V'| \geq k$.

$\therefore$ VERTEX-COVER is NP-complete.

Key Idea

VERTEX-COVER is NP-complete — yet a simple polynomial-time *approximation* algorithm exists that always returns a cover at most $2\times$ the optimal size (Ch. 35, next unit).

Hold that thought — it's exactly the kind of fallback strategy we'll formalize in the next section.

NP-Hardness in Practice

Section 7 — 1:16 – 1:33

A quick diagnostic checklist

- 1 Does the problem ask you to choose an *optimal* subset, subgraph, ordering, or assignment from a search space that grows exponentially with input size?
- 2 Does brute force look like 2^n , $n!$, or $\binom{n}{k}$ with $k \sim n$ — and has no polynomial algorithm turned up despite genuine literature search?
- 3 Can you sketch — even informally — a reduction from a problem you already know is NP-complete (SAT, CLIQUE, VERTEX-COVER, TSP, KNAPSACK, SET-PARTITION. . .) into your problem?

Key Idea

You rarely need a textbook-grade proof on the job. A convincing *sketch* of a reduction is often enough to justify changing strategy — that's the standard of evidence working engineers actually use.

Scheduling

- Task/CPU scheduling with deadlines and profits
- Exam / room timetabling under conflicts

Network & hardware design

- Graph coloring → register allocation, VLSI layout
- Steiner tree → network / circuit routing

Logistics

- Vehicle routing / delivery scheduling (TSP family)

Systems & infra

- Cloud resource allocation / bin packing
- Query plan / join-order optimization

None of these industries wait for $P = NP$ to ship products. They all use the same handful of fallback strategies — next slide.

- 1 **Look for special-case structure** in your actual input — bounded degree, bipartiteness, bounded treewidth, sparsity — that a restricted poly-time algorithm can exploit.
- 2 **Approximation algorithms** with a *provable* quality bound (e.g., VERTEX-COVER's 2-approximation).
- 3 **Heuristics / metaheuristics** without formal guarantees — greedy, local search, simulated annealing — when “good enough, fast” beats “optimal, slow.”
- 4 **Exponential-but-practical solvers** — branch-and-bound, ILP solvers, modern SAT solvers — which are often fast in practice despite worst-case exponential behavior.
- 5 **Reformulate the question** — restrict the decision version's bound, or look for fixed-parameter tractability in some small parameter k of your actual workloads.

Independent Set — NP-complete in general, easy in special cases

- **General graphs:** NP-complete (reduces from CLIQUE — an independent set in G is a clique in $\overline{G}$).
- **Degree-bounded graphs** (e.g. every vertex has degree ≤ 2): solvable directly in polynomial time — the graph decomposes into simple paths and cycles.
- **Bipartite graphs:** solvable in polynomial time via a maximum-matching algorithm (König's theorem connects independent-set size to matching size).

Key Idea

Before reaching for an approximation or heuristic, always ask: *does my real workload actually look like the general worst case, or does it have exploitable structure?*

IUT exam-scheduling scenario

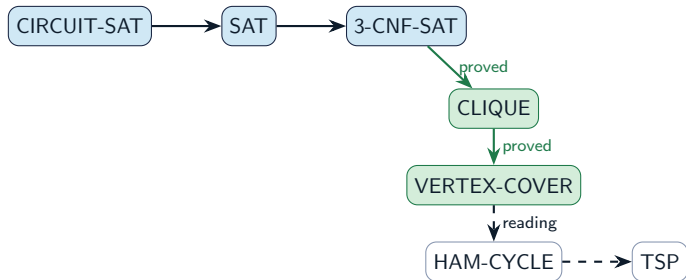
You must assign 500 exams to time slots such that no student has two exams in conflicting slots, using as few slots as possible.

- 1 Which NP-complete problem does this resemble, and why?
- 2 What's a defensible *first* reduction sketch you'd offer in a design review to justify calling it NP-hard?
- 3 Given the checklist above, which fallback strategy would you reach for first — and what property of the *actual* exam-conflict data would make you choose it?

(2–3 min discussion — be ready to defend your answer verbally.)

Wrap-Up

Section 8 — 1:33 – 1:40



- **P vs. NP:** verifying is (believed) fundamentally easier than solving.
- **Reductions** are the load-bearing tool of the whole theory — both to prove hardness and to transfer solutions.
- **NP-completeness** is community-grade evidence of intractability, not a proof — and that distinction matters for how you act on it as an engineer.

- VERTEX-COVER $\rightarrow$ HAM-CYCLE and HAM-CYCLE $\rightarrow$ TSP (gadget constructions, CLRS §34.5.3–34.5.4) — optional, for students who want to see the gadget-design technique in full.
- SUBSET-SUM reduction from 3-CNF-SAT (CLRS §34.5.5) — a good second example of “crossing domains” in a reduction.
- Reduction *strategy* recap (CLRS §34.5.6): avoid backward reductions, go general-to-specific, exploit structure in the source problem, look for special cases, choose gadgets deliberately.

For your next lab / defense

Expect a “classify and justify” style prompt: given an unfamiliar problem, argue — verbally, on the spot — whether it is NP-hard, using a reduction *sketch* rather than a full formal proof. That is the standard we practiced today.

Next unit

Chapter 35: Approximation Algorithms — what to do *after* you’ve established a problem is NP-hard and still need an answer.

Questions?